

UNIT III

LEETCODE EXERCISE

1.Combine Two Tables

Table: Person

Column Name	Type
PersonId	int
FirstName	varchar
LastName	varchar

PersonId is the primary key column for this table.

Table: Address

Column Name	Type
AddressId	int
PersonId	int
City	varchar
State	varchar

AddressId is the primary key column for this table.

Q: Write a SQL query for a report that provides the following information for each person in the Person table, regardless if there is an address for each of those people:
FirstName, LastName, City, State

Solution:

```
1 # Write your MySQL query statement below
2 select FirstName, LastName, City, State
3 from Person left join Address
```

Testcase | Test Result

Accepted Runtime: 100 ms

Case 1

Input

Person =

personId	lastName	firstName
1	Wang	Allen
2	Alice	Bob

Address =

addressId	personId	city	state
1	2	New York City	New York
2	3	Leetcode	California

Output

FirstName	LastName	City	State
Allen	Wang	null	null
Bob	Alice	New York City	New York

Explanation:

LEFT JOIN returns all persons. If no address is found, City and State are shown as NULL.

2. Employees Earning More Than Their Managers

The Employee table holds all employees including their managers. Every employee has an Id, and there is also a column for the manager Id.

Id	Name	Salary	ManagerId
1	Joe	70000	3
2	Henry	80000	4
3	Sam	60000	NULL
4	Max	90000	NULL

Given the Employee table, write a SQL query that finds out employees who earn more than their managers.

For the above table, Joe is the only employee who earns more than his manager.

Employee
Joe

Q: Write a SQL query that finds out employees who earn more than their managers. For the above table, Joe is the only employee who earns more than his manager.

Solution:

```

1  # Write your MySQL query statement below
2  select a.name as employee
3  from Employee a
4  left join Employee b
5  on (a.managerid = b.id)
6  where (a.salary > b.salary)

```

Testcase | Test Result

Case 1

Input

Employee =

id	name	salary	managerId
1	Joe	70000	3
2	Henry	80000	4
3	Sam	60000	null
4	Max	90000	null

Output

employee
Joe

Explanation:

The query compares employee and manager salaries. It selects employees earning more than their managers. Joe earns more than his manager Sam.

3.Customer Who Visited but Did Not Make Any Transactions

Table: Visits

Column Name	Type
visit_id	int
customer_id	int

visit_id is the column with unique values for this table.

This table contains information about the customers who visited the mall.

Table: Transactions

Column Name	Type
transaction_id	int
visit_id	int
amount	int

transaction_id is column with unique values for this table.

This table contains information about the transactions made during the visit_id.

Return the result table sorted in any order.

The result format is in the following example.

Q: Write a solution to find the IDs of the users who visited without making any transactions and the number of times they made these types of visits.

Solution:

```
1 # Write your MySQL query statement below
2 SELECT v.customer_id,COUNT(*) AS count_no_trans
3 FROM Visits v
4 LEFT JOIN Transactions t ON v.visit_id = t.visit_id
5 WHERE t.transaction_id IS NULL GROUP BY v.customer_id;
```

Testcase | > Test Result

Input

Visits =

visit_id	customer_id
1	23
2	9
4	30
5	54
6	96
7	54
8	54

Transactions =

transaction_id	visit_id	amount
2	5	310
3	5	300
9	5	200
12	1	910
13	2	970

Output

customer_id	count_no_trans
30	1
96	1
54	2

Explanation:

Customer with id = 23 visited the mall once and made one transaction during the visit with id = 12.

Customer with id = 9 visited the mall once and made one transaction during the visit with id = 13.

Customer with id = 30 visited the mall once and did not make any transactions.

Customer with id = 54 visited the mall three times.

During 2 visits they did not make any transactions, and during one visit they made 3 transactions.

Customer with id = 96 visited the mall once and did not make any transactions.

As we can see, users with IDs 30 and 96 visited the mall one time without making any transactions.

Also, user 54 visited the mall twice and did not make any transactions.

4. Rising Temperature

Table: Weather

Column Name	Type
id	int
recordDate	date
temperature	int

id is the primary key for this table.

This table contains information about the temperature in a certain day.

Q:Write an SQL query to find all dates id with higher temperature compared to its previous dates (yesterday).Return the result table in any order.

Solution:

 Code

MySQL   Auto

```
1 select w1.id
2 from Weather w1, Weather w2
3 where (w1.Temperature > w2.Temperature and
4      datediff(w1.recordDate, w2.recordDate) = 1)
```

Testcase | > Test Result

Accepted Runtime: 70 ms

✓ Case 1

Input

Weather =

id	recordDate	temperature
1	2015-01-01	10
2	2015-01-02	25
3	2015-01-03	20
4	2015-01-04	30

Output

id
2
4

Explanation:

The query compares each day's temperature with the temperature of the previous day using recordDate. It returns the dates where the temperature is higher than the previous day.

5.Managers with at Least 5 Direct Reports

Table: Employee

Column Name	Type
id	int
name	varchar
department	varchar
managerId	int

id is the primary key (column with unique values) for this table.
Each row of this table indicates the name of an employee, their department, and the id of their manager.
If managerId is null, then the employee does not have a manager.
No employee will be the manager of themselves.
Return the result table in any order.

Q: Write a solution to find managers with at least five direct reports.

Solution:

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT name
3 FROM Employee
4 WHERE id IN
5     (SELECT managerid
6      FROM Employee
7      GROUP BY managerid
8      HAVING COUNT(id) >= 5)
```


✓ Testcase | >_ Test Result

Accepted Runtime: 76 ms

✓ Case 1

Input

Employee =

id	name	department	managerId
101	John	A	null
102	Dan	A	101
103	James	A	101
104	Amy	A	101
105	Anne	A	101
106	Ron	B	101

Output

name
John

Explanation:

The query groups employees based on their managerId and counts their direct reports. It returns the names of managers who have 5 or more employees reporting to them.